

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ  
АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ  
УНИВЕРСИТЕТ ИТМО»

ФАКУЛЬТЕТ ТЕХНОЛОГИЙ ИСКУССТВЕННОГО ИНТЕЛЛЕКТА



---

Отчёт по лабораторной работе №2  
«Взаимодействие с внешним API»  
по дисциплине «Web-разработка: Frontend»  
Семестр 4

---

**Выполнил:** студент  
Мавров Артём Николаевич  
гр. J3213  
ИСУ 466574

**Отчёт сдан:** 30.03.2026

**Проверил:** Добряков Д. И.

Санкт-Петербург  
2026

# Содержание

|          |                                                 |          |
|----------|-------------------------------------------------|----------|
| <b>1</b> | <b>Задача</b>                                   | <b>3</b> |
| <b>2</b> | <b>Ход работы</b>                               | <b>3</b> |
| 2.1      | Структура проекта . . . . .                     | 3        |
| 2.2      | База данных JSON Server . . . . .               | 3        |
| 2.3      | Сервер авторизации . . . . .                    | 3        |
| 2.4      | Fetch-клиент api.js . . . . .                   | 5        |
| 2.5      | Подключение авторизации в app.js . . . . .      | 5        |
| 2.6      | Перевод операций с данными на fetch . . . . .   | 6        |
| 2.7      | Индикатор загрузки и обработка ошибок . . . . . | 7        |
| <b>3</b> | <b>Вывод</b>                                    | <b>8</b> |

## Задача

В рамках данной лабораторной работы необходимо подключить к уже реализованному сайту платформы управления ML-пайплайнами моковое API.

## Ход работы

### Структура проекта

В результате выполнения работы файловая структура проекта расширилась:

- `server.js` — кастомный сервер на базе JSON Server с middleware авторизации;
- `db.json` — файл базы данных (пользователи, эксперименты, модели, артефакты, сессии);
- `api.js` — fetch-клиент, единая точка обращения к API;
- `app.js` — логика приложения, обновлена для работы с API;

### База данных JSON Server

Файл `db.json` содержит все коллекции приложения в виде JSON-массивов. Помимо основных данных, в нём хранится коллекция `sessions` — активные токены пользователей:

```
1 {  
2   "users":      [...],  
3   "experiments": [...],  
4   "models":     [...],  
5   "artifacts":  [...],  
6   "sessions":   []  
7 }
```

Листинг 1: Фрагмент структуры `db.json`

JSON Server автоматически предоставляет REST-интерфейс для каждой коллекции: GET, POST, PUT, PATCH, DELETE. Кастомный `server.js` расширяет его собственными маршрутами авторизации.

### Сервер авторизации

Сервер реализован через `jsonServer.create()` с подключением собственных middleware. Добавлены четыре маршрута:

- POST `/auth/login` — поиск пользователя в базе, генерация токена и сохранение сессии;
- POST `/auth/register` — создание нового пользователя и сессии;
- POST `/auth/logout` — удаление сессии по токену;
- GET `/auth/me` — получение данных текущего пользователя по токену (используется для восстановления сессии).

Логика входа в систему:

```
1 server.post('/auth/login', function (req, res) {
2   var email    = (req.body.email    || '').trim();
3   var password = (req.body.password || '');
4   var db      = router.db.getState();
5   var user = db.users.find(function(u) {
6     return u.email === email && u.password === password;
7   });
8   if (!user) {
9     return res.status(401).json(
10      { error: 'Неверная email или пароль', }
11    );
12  }
13  var token = 'mlp_'
14    + Math.random().toString(36).slice(2)
15    + Date.now().toString(36);
16  // db.json
17  var sessions = db.sessions.filter(
18    function(s) { return s.userId !== user.id; }
19  );
20  sessions.push({
21    token: token,
22    userId: user.id,
23    createdAt: new Date().toISOString()
24  });
25  router.db.set('sessions', sessions).write();
26  return res.status(200).json({ token: token, user: user });
27 });
```

Листинг 2: Обработчик POST /auth/login в server.js

Для защиты маршрутов с данными добавлен middleware, который проверяет токен из заголовка Authorization: Bearer <token> перед каждым запросом к /experiments, /models, /artifacts:

```
1 server.use(function (req, res, next) {
2   var PROTECTED = ['/experiments', '/models', '/artifacts'];
3   var isProtected = PROTECTED.some(function(path) {
4     return req.path.startsWith(path);
5   });
6   if (!isProtected) return next();
7
8   var token = (req.headers['authorization'] || '')
9     .replace('Bearer ', '').trim();
10  var session = db.sessions.find(
11    function(s) { return s.token === token; }
12  );
13  if (!session) {
14    return res.status(401).json(
15      { error: 'Неверный токен', }
16    );
17  }
18  next();
19 });
```

## Fetch-клиент api.js

Все HTTP-запросы инкапсулированы в модуле `api.js`. Базовая функция `apiFetch` автоматически подставляет токен в заголовок и обрабатывает ошибки:

```

1 function apiFetch(method, path, body) {
2   var headers = { 'Content-Type': 'application/json' };
3   if (_token) {
4     headers['Authorization'] = 'Bearer ' + _token;
5   }
6   var opts = { method: method, headers: headers };
7   if (body) opts.body = JSON.stringify(body);
8
9   return fetch(API_BASE + path, opts)
10    .then(function(res) {
11      if (res.status === 204) return null;
12      return res.json().then(function(data) {
13        if (!res.ok) throw new Error(
14          data.error || ' ' + res.status
15        );
16        return data;
17      });
18    });
19 }
```

Листинг 4: Базовая обёртка над fetch в api.js

Токен хранится в переменной и дублируется в `sessionStorage`, что позволяет восстановить авторизацию при перезагрузке страницы:

```

1 var _token = sessionStorage.getItem('mlpipe_token') || null;
2
3 function setToken(t) {
4   _token = t;
5   if (t) sessionStorage.setItem('mlpipe_token', t);
6   else    sessionStorage.removeItem('mlpipe_token');
7 }
```

Листинг 5: Хранение токена сессии

На основе `apiFetch` реализованы методы для каждой сущности: `API.login()`, `API.getExperi`, `API.createModel()`, `API.deleteArtifact()` и другие.

## Подключение авторизации в app.js

При загрузке страницы приложение проверяет наличие сохранённого токена и восстанавливает сессию без повторного ввода пароля:

```

1 document.addEventListener('DOMContentLoaded', function () {
2   if (getToken()) {
3     showLoader(true);
4     API.me()
```

```

5      .then(function(user) {
6          currentUser = user;
7          startApp();
8      })
9      .catch(function() {
10         setToken(null); //
11         showLoader(false);
12     });
13 }
14 });

```

Листинг 6: Восстановление сессии при загрузке страницы

Функция `doLogin()` теперь вызывает `API.login()`, а не ищет пользователя в локальном массиве. При успехе токен сохраняется, при ошибке — сообщение выводится прямо из ответа сервера:

```

1 function doLogin() {
2     var email = document.getElementById('li-email').value.trim();
3     var pass   = document.getElementById('li-pass').value;
4
5     API.login(email, pass)
6         .then(function(user) {
7             currentUser = user;
8             startApp();
9         })
10        .catch(function(e) {
11            err.textContent = e.message || ' ';
12            err.classList.remove('d-none');
13        });
14 }

```

Листинг 7: Авторизация через fetch

## Перевод операций с данными на fetch

Все операции чтения и записи переведены на асинхронные вызовы API. Например, загрузка экспериментов при переходе в раздел:

```

1 function loadExperiments() {
2     showLoader(true);
3     API.getExperiments()
4         .then(function(data) {
5             experiments = data;
6             renderExps(experiments);
7         })
8         .catch(function(e) { showApiError(e.message); })
9         .finally(function() { showLoader(false); });
10 }

```

Листинг 8: Загрузка экспериментов через fetch

Мутирующие операции — `promote`, `deploy`, `archive` — также отправляют запросы к серверу и обновляют локальный массив только после успешного ответа:

```

1 function promoteModel(id) {

```

```

2   var order = ['dev', 'staging', 'prod'];
3   var m = models.find(function(x) { return x.id === id; });
4   var newStage = order[order.indexOf(m.stage) + 1];
5
6   API.updateModel(id, { stage: newStage })
7     .then(function(updated) {
8       var i = models.findIndex(
9         function(x) { return x.id === id; }
10      );
11      if (i !== -1) models[i] = updated;
12      renderModels();
13    })
14    .catch(function(e) { showApiError(e.message); });
15 }

```

Листинг 9: Продвижение модели по стадиям через PATCH

Удаление артефакта:

```

1 function deleteArtifact(id) {
2   API.deleteArtifact(id)
3     .then(function() {
4       artifacts = artifacts.filter(
5         function(a) { return a.id !== id; }
6       );
7       renderArtifacts(artifacts);
8     })
9     .catch(function(e) { showApiError(e.message); });
10 }

```

Листинг 10: Удаление артефакта через DELETE

## Индикатор загрузки и обработка ошибок

В интерфейс добавлены два элемента обратной связи. Оверлей загрузки показывается на время каждого API-запроса:

```

1 <div id="api-loader" style="display:none;
2   position:fixed; inset:0;
3   background:rgba(8,11,16,.6); z-index:9999;
4   align-items:center; justify-content:center;">
5   <div class="spinner-border text-info"></div>
6 </div>

```

Листинг 11: HTML-разметка индикатора загрузки

Toast-уведомление показывает текст ошибки от сервера и автоматически скрывается через 4 секунды:

```

1 function showApiError(msg) {
2   var el = document.getElementById('api-error');
3   el.textContent = msg;
4   el.style.display = 'block';
5   setTimeout(function() {
6     el.style.display = 'none';
7   }, 4000);

```

## Листинг 12: Функция вывода ошибки API

## Вывод

В ходе лабораторной работы сайт платформы управления ML-пайплайнами был подключён к моковому API на базе JSON Server. Реализован кастомный сервер с маршрутами авторизации (`/auth/login`, `/auth/register`, `/auth/me`) и middleware защиты маршрутов по токену.

Все операции с данными переведены с локальных массивов на асинхронные `fetch`-запросы. Токен сессии сохраняется в `sessionStorage` и восстанавливается при перезагрузке страницы. Добавлены индикатор загрузки и toast-уведомления об ошибках API.